

学术论文搜索与推荐系统

Academic Paper Search & Recommend System

数据库引论——项目报告

课程名称： 数据库引论实验课

选题方向： 论文搜索与推荐

姓 名： 年骏杰

学 号： 24307140012

提交日期： 2026 年 6 月 1 日

目录

1	项目概述	3
1.1	项目背景	3
1.2	选题说明	3
1.3	技术栈总览	4
2	系统架构设计	4
2.1	分层架构	4
2.2	数据流设计	5
3	数据库设计	6
3.1	ER 图	6
3.2	表结构详细设计	7
3.2.1	users —用户表	7
3.2.2	papers —论文表	7
3.2.3	user_clicks —点击记录表	8
3.2.4	search_history —搜索历史表	8
3.3	关系说明	8
4	业务层实现	8
4.1	用户认证	8
4.2	API 接口设计	10
4.3	核心业务逻辑	11
4.3.1	搜索代理与降级	11
4.3.2	推荐代理	11
4.3.3	对话上下文注入与 RAG	11
4.3.4	论文上传与 arXiv 导入	11
5	算法层实现	12
5.1	向量语义检索	12
5.1.1	索引构建	12
5.1.2	检索流程	12
5.2	基于 Embedding 质心的推荐算法	12
5.3	Reranker 精排	13
5.4	PDF 全文提取与分块 Embedding	13
5.4.1	全文提取	13
5.4.2	分块策略	13
5.4.3	向量索引	13
5.5	LLM 学术问答与 RAG	14

5.5.1	论文问答	14
5.5.2	RAG 全库检索增强生成	14
6	前端界面实现	14
6.1	页面组成	15
6.2	关键前端特性	15
6.3	路由设计	16
7	算法分析与实验	16
7.1	实验环境	16
7.2	评估指标	17
7.3	实验一：搜索系统消融实验	17
7.4	实验二：推荐系统消融实验	18
7.5	实验三：搜索系统性能评估	19
7.6	实验四：推荐系统性能评估	20
7.7	实验五：距离度量对比	20
7.8	实验六：文档表示对比	20
7.9	实验七：Reranker 对比	21
7.10	实验八：数据规模扩展测试	22
7.11	实验小结	22
8	测试数据	23
8.1	论文数据集	23
8.2	Ground Truth 构建	23
8.3	推荐评估数据	23
9	用户手册	24
9.1	环境要求	24
9.2	安装与配置	24
9.3	启动服务	25
9.4	导入种子数据	25
9.5	功能使用	25
9.6	访问地址	26
9.7	运行实验	26
10	项目结构	26
11	总结	27

1 项目概述

1.1 项目背景

随着学术论文数量的爆炸式增长，研究者在海量文献中高效检索并发现有价值的论文成为一项重要需求。传统的关键词匹配搜索仅能捕获字面重合，无法理解查询的语义意图。本项目面向数据库与数据管理领域，构建了一套融合**向量语义检索**、**基于 Embedding 质心的个性化推荐**、**可选精排**、**PDF 全文提取与分块 Embedding**、**基于 RAG 的大语言模型学术问答**以及**用户论文上传（arXiv 链接一键导入）**的全栈论文搜索与推荐系统。用户既可检索系统预置的公共论文库，也可上传自己关注的论文，上传后的论文自动进入个人专属库并参与语义搜索、个性化推荐与 AI 问答。

1.2 选题说明

本项目选题为”**论文搜索与推荐**”：根据用户输入的自然语言查询，从论文库中语义搜索论文；根据用户的点击浏览历史，个性化推荐相关论文；同时集成 LLM 多轮对话，辅助用户理解论文内容。在此基础上进一步扩展了**用户自助上传论文能力**——用户粘贴 arXiv 链接即可由系统自动提取元数据、下载 PDF 并建立索引，使个人论文同样可被搜索、推荐与问答（仅上传者本人可见）。之所以采用**链接导入**而非直接上传 PDF 文件，是因为搜索与推荐的质量高度依赖规范、完整的结构化元数据（标题、摘要、关键词等）；链接导入可从权威来源（OpenAlex/arXiv）自动获取这些元数据与原文 PDF，既免去用户手工录入、避免字段缺失或格式不一，又保证了语义向量化所依赖的文本质量。

1.3 技术栈总览

表 1: 系统技术栈

层级	技术	版本	说明
前端 (界面层)	Vue 3	3.4.21	Composition API
	Element Plus	2.6.3	UI 组件库
	Pinia	2.1.7	状态管理
	Vite	5.1.6	构建工具
业务层	FastAPI	0.114.0	异步 Web 框架
	SQLAlchemy	2.0.35	ORM
	PyJWT	2.8.0	JWT 认证
	Passlib+bcrypt	—	密码哈希
算法层	FastAPI	0.114.0	独立微服务
	ChromaDB	—	向量数据库
	NumPy	—	数值计算
	PyMuPDF (fitz)	1.27	PDF 全文提取
存储	MySQL 8.0+	—	关系型数据库
	ChromaDB	—	向量索引持久化
模型	text-embedding-v4	1024 维	语义向量化
	qwen-flash	—	LLM 对话
	qwen3-rerank	—	可选精排

2 系统架构设计

2.1 分层架构

系统严格遵循课程要求的**三层分离架构**：界面层负责与用户交互，业务层处理逻辑与数据持久化，算法层专注核心搜索/推荐/对话算法。各层通过 HTTP RESTful API 通信，可独立部署与扩展。

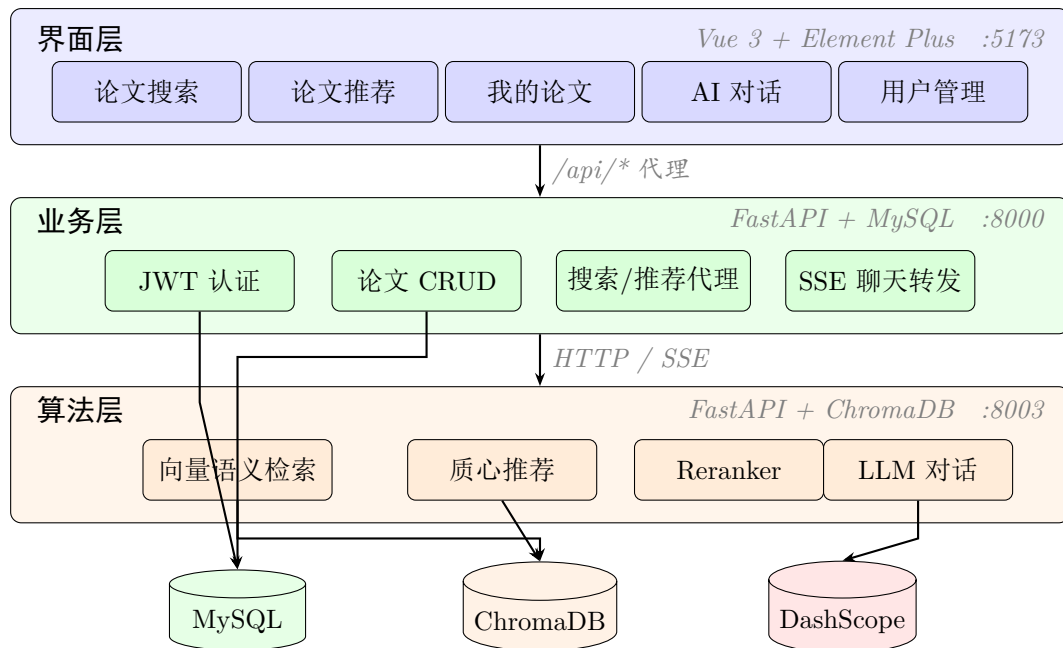


图 1: 系统三层架构图

2.2 数据流设计

系统的三条核心数据流如下：

- 搜索流程：**用户输入查询 → 业务层记录搜索历史至 MySQL → 转发至算法层 → 查询文本经 `text-embedding-v4` 向量化 → ChromaDB ANN 检索 → 可选 Reranker 精排 → 返回 `paper_id + score` → 业务层从 MySQL 补全论文元数据 → 分页返回前端。
- 推荐流程：**用户浏览论文时前端调用 `/click` 接口记录点击 → 请求推荐时业务层获取用户最近 50 条点击 → 算法层从 ChromaDB 获取已点击论文的 Embedding → 计算质心向量 → ChromaDB 近邻搜索 → 过滤已读 → 返回 Top-K 推荐。
- 对话流程（三种模式）：**
 - 论文问答模式：**论文详情页提问 → 业务层调用算法层 `/search-chunks` 按 `paper_id` 检索该论文全文分块 → 注入论文元信息与全文片段 → LLM 基于全文内容回答。
 - RAG 全库模式：**AI 对话页提问 → 业务层调用 `/search-chunks` 全库检索最相关分块 → 按论文聚合并注入上下文 → LLM 回答并附带参考论文列表 → 前端显示可点击的引用标签。
 - 纯对话模式：**注入用户最近 5 篇已读论文摘要 → 转发至算法层 → SSE 流式返回。
- 全文索引流程：**PDF 全文经 PyMuPDF 提取 → 按约 800 词/块分块（100 词重叠） → 批量调用算法层 `/index-chunks` → Embedding 后存入 ChromaDB `paper_chunks` collection。
- 论文上传与导入流程：**用户在“我的论文”页粘贴 arXiv 链接 → 业务层解析

arXiv ID 并调用 OpenAlex API 提取规范元数据（标题、作者、摘要、年份、关键词）→ 用户确认/微调后提交 → 业务层将论文写入 MySQL（以 `owner_id` 标记归属）并从 arxiv.org 下载 PDF 原文 → 同时调用算法层/`index`（摘要级向量，供搜索/推荐）与/`index-chunks`（全文分块，供问答/RAG）建立双重索引，使上传论文即刻可被检索与问答。

可见性隔离: `papers` 表通过 `owner_id` 字段区分论文归属——系统预置论文 `owner_id` 为 NULL，对所有用户公共可见；用户上传论文 `owner_id` 为其用户 ID，仅本人可见。业务层在搜索、推荐、RAG 检索以及论文详情/PDF/问答各环节，统一按“`owner_id IS NULL OR owner_id = 当前用户`”过滤，确保他人的私有论文不会出现在自己的任何检索结果中。

降级机制: 当算法层不可用时，搜索自动降级为 MySQL LIKE 模糊匹配（同样施加上述可见性过滤），保障系统基本可用性。

3 数据库设计

3.1 ER 图

系统包含 4 张核心表，通过外键关联形成完整的数据模型。

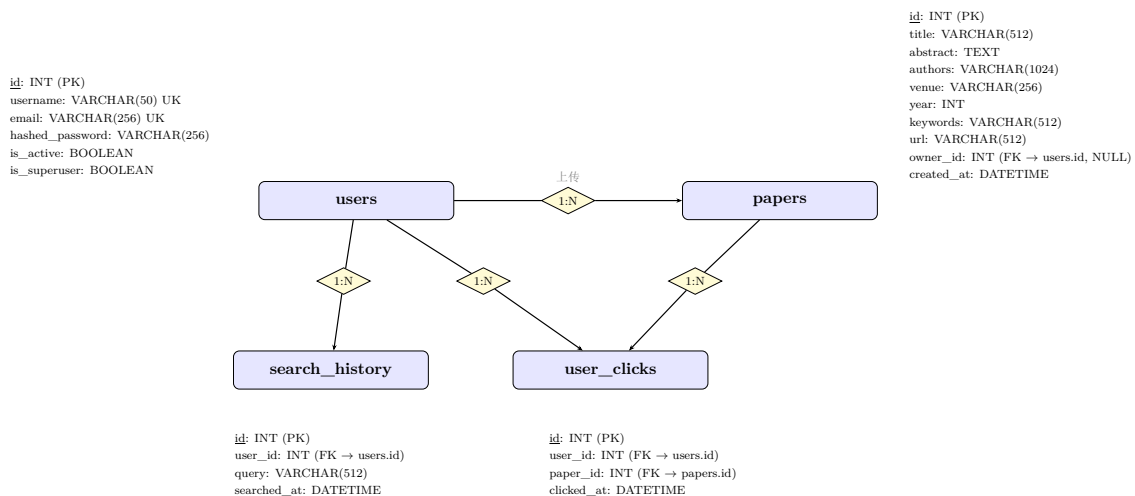


图 2: 系统 ER 图

3.2 表结构详细设计

3.2.1 users 一用户表

表 2: users 表结构

字段	类型	约束	说明
id	INT	PK, AUTO_INCREMENT	主键
username	VARCHAR(50)	UNIQUE, NOT NULL	登录用户名
email	VARCHAR(256)	UNIQUE, INDEX	邮箱地址
first_name	VARCHAR(128)		名
last_name	VARCHAR(128)		姓
hashed_password	VARCHAR(256)		bcrypt 加密密码
is_active	BOOLEAN	DEFAULT TRUE	账户激活状态
is_superuser	BOOLEAN	DEFAULT FALSE	管理员标志

3.2.2 papers 一论文表

表 3: papers 表结构

字段	类型	约束	说明
id	INT	PK, AUTO_INCREMENT	主键
title	VARCHAR(512)		论文标题
abstract	TEXT		摘要
authors	VARCHAR(1024)		作者（逗号分隔）
venue	VARCHAR(256)		发表会议/期刊
year	INT		发表年份
keywords	VARCHAR(512)		关键词（逗号分隔）
url	VARCHAR(512)		论文来源链接
owner_id	INT	FK → users.id, NULL, INDEX	上传者；NULL 表示系统公共论文
created_at	DATETIME	DEFAULT UTC_NOW	入库时间

3.2.3 user_clicks 一点击记录表

表 4: user_clicks 表结构

字段	类型	约束	说明
id	INT	PK, AUTO_INCREMENT	主键
user_id	INT	FK → users.id	点击用户
paper_id	INT	FK → papers.id	被点击论文
clicked_at	DATETIME	DEFAULT UTC_NOW	点击时间

3.2.4 search_history 一搜索历史表

表 5: search_history 表结构

字段	类型	约束	说明
id	INT	PK, AUTO_INCREMENT	主键
user_id	INT	FK → users.id	搜索用户
query	VARCHAR(512)		查询文本
searched_at	DATETIME	DEFAULT UTC_NOW	搜索时间

3.3 关系说明

- users 1:N user_clicks——一个用户可有多条点击记录。
- papers 1:N user_clicks——一篇论文可被多个用户点击。
- users 1:N search_history——一个用户可有多条搜索历史。
- users 1:N papers（上传关系）——一个用户可上传多篇论文（papers.owner_id 指向 users.id）；owner_id 为 NULL 的论文是系统预置的公共论文，不属于任何特定用户。
- user_clicks 构成 users 与 papers 之间的多对多关系（附带时间戳的关联表）。

4 业务层实现

业务层基于 FastAPI 框架（端口 8000），通过 SQLAlchemy ORM 操作 MySQL，负责用户认证、论文 CRUD、搜索/推荐代理以及 LLM 对话转发。

4.1 用户认证

系统采用 OAuth2 Bearer Token + JWT 方案：

1. 用户通过 POST /token 提交用户名和密码。

2. 后端使用 `bcrypt` 验证密码哈希，通过后生成 JWT Token（HS256 签名，有效期 30 分钟）。
3. 后续请求在 `Authorization: Bearer <token>` 头中携带 Token。
4. 受保护接口通过 `oauth2_scheme` 依赖注入自动解析并验证 Token。

4.2 API 接口设计

表 6: 业务层 API 接口

端点	方法	认证	说明
认证与用户			
/token	POST	否	登录, 返回 JWT
/users/	POST	否	注册
/users/me/	GET	是	当前用户信息
/users/	GET	是	用户列表 (分页)
/users/{id}	GET	是	按 ID 获取用户
/users/name/{name}	GET	是	按用户名获取
论文管理			
/papers/	GET	是	论文列表 (分页)
/papers/{id}	GET	是	论文详情 (按 owner 过滤)
/papers/mine	GET	是	我上传的论文列表
/papers/bulk	POST	是	批量导入
论文上传 (arXiv 导入)			
/papers/preview-arxiv	POST	是	解析 arXiv 链接, OpenAlex 提取元数据 (不落库)
/papers/import-arxiv	POST	是	导入论文: 下载 PDF + 双重索引
/papers/{id}	DELETE	是	删除自己上传的论文 (含向量/PDF)
搜索与推荐			
/search	POST	是	语义搜索 + 降级 SQL
/click	POST	是	记录论文点击
/recommend	GET	是	个性化推荐
/search/history	GET	是	搜索历史
AI 对话			
/chat	POST	是	非流式 LLM 对话
/chat/stream	POST	是	流式 SSE 对话 (论文问答/RAG/纯对话)
PDF 管理			
/papers/{id}/pdf	GET	是	下载论文 PDF
/papers/{id}/upload-pdf	POST	是	上传 PDF (管理员)

4.3 核心业务逻辑

4.3.1 搜索代理与降级

POST /search 接口接收用户查询后：

1. 将查询记录写入 search_history 表。
2. 将请求转发至算法层/search 端点，附带 query、use_rerank、rerank_provider、recall_k 等参数。
3. 算法层返回 paper_id + score 列表后，业务层从 MySQL 批量查询论文元数据，并按 owner_id 可见性（公共论文 + 本人上传）过滤后再分页拼接。
4. 若算法层不可用，则降级为 SQL LIKE 搜索（匹配 title、abstract、keywords、authors 字段，同样附加 owner_id 可见性过滤）。

4.3.2 推荐代理

GET /recommend 接口：

1. 从 user_clicks 表查询当前用户最近 50 条点击记录。
2. 将已点击论文 ID 列表发送至算法层/recommend 端点。
3. 算法层返回推荐结果后，业务层从 MySQL 补全元数据返回前端。

4.3.3 对话上下文注入与 RAG

POST /chat/stream 接口支持三种模式，由请求参数 paper_id 和 use_rag 控制：

1. 论文问答模式（paper_id 有值）：调用算法层/search-chunks 检索该论文全文分块（Top-5），将论文元信息与全文片段注入系统提示词，LLM 基于论文实际内容回答。
2. RAG 全库模式（use_rag=true,默认）：以用户最后一条消息为查询，调用/search-chunks 检索全库最相关的 8 个分块，按 paper_id 聚合后注入上下文，同时构建引用列表。SSE 流结束后发送 references 事件，前端显示可点击的参考论文标签。
3. 纯对话模式（use_rag=false）：保留原有逻辑，仅注入用户最近 5 篇已读论文摘要。

4.3.4 论文上传与 arXiv 导入

为支持用户导入自己关注的论文，业务层新增了 paper_ingest 模块与三个接口，完整流程分两步：

1. 解析预览（POST /papers/preview-arxiv）：从用户粘贴的链接中用正则解析出 arXiv ID（兼容/abs/、/pdf/、带版本号 vN、旧式 cs/0112017 等形式），再以 arXiv 的 DataCite DOI（10.48550/arXiv.<id>）调用 OpenAlex API 查询元数据，重建摘要（倒排索引还原）、截断作者与关键词后返回前端供用户确认与编辑；若 OpenAlex 未收录则返回空预览，由用户手动补全。

2. **确认导入** (POST /papers/import-arxiv): 将用户确认后的元数据写入 MySQL (owner_id= 当前用户, 先 flush 取得自增主键但暂不提交) → 从 arxiv.org/pdf/<id> 下载 PDF 原文并校验%PDF 魔数, 失败则回滚 → 成功后提交并调用算法层/index (摘要级向量) 与/index-chunks (PyMuPDF 提取全文并分块) 建立双重索引。

删除接口 (DELETE /papers/{id}) 仅允许删除本人上传的论文, 会级联清除本地 PDF 文件、两个 ChromaDB collection 中的向量 (经算法层/delete-paper) 以及 MySQL 记录与点击历史。

数据源选择: 实测部署环境下 arXiv 官方元数据 API (export.arxiv.org) 连通性不稳定, 而 OpenAlex API 稳定可达、arxiv.org 的 PDF 可直接下载, 故采用”OpenAlex 取元数据 + arxiv.org 取 PDF” 的组合, 兼顾元数据规范性与原文可得性。

5 算法层实现

算法层作为独立的 FastAPI 微服务 (端口 8003), 封装了向量检索、推荐和对话三大核心算法。

5.1 向量语义检索

5.1.1 索引构建

对每篇论文 p_i , 拼接其标题、摘要和关键词构成文档文本, 通过 text-embedding-v4 模型生成 1024 维向量, 存入 ChromaDB:

$$\text{doc}_i = \text{concat}(p_i.\text{title}, ". ", p_i.\text{abstract}, ". ", p_i.\text{keywords}) \quad (1)$$

$$\mathbf{e}_i = \text{text-embedding-v4}(\text{doc}_i) \in \mathbb{R}^{1024} \quad (2)$$

Embedding 采用批量请求 (每批最多 10 条), 通过 DashScope OpenAI 兼容接口调用。

5.1.2 检索流程

给定查询文本 q 和返回数量 k :

1. 将查询向量化: $\mathbf{q} = \text{text-embedding-v4}(q)$
2. ChromaDB ANN 近邻搜索, 返回 Top- k 结果及距离 d_i
3. 将距离转换为相关性分数: $\text{score}_i = \frac{1}{1+d_i}$
4. 按分数降序返回

5.2 基于 Embedding 质心的推荐算法

给定用户 u 的点击历史 $C = \{c_1, c_2, \dots, c_m\}$ (最近 50 篇) 和推荐数量 k :

1. 从 ChromaDB 获取已点击论文的 Embedding: $E = \{\mathbf{e}(c_1), \dots, \mathbf{e}(c_m)\}$

2. 计算兴趣质心：

$$\mathbf{centroid} = \frac{1}{m} \sum_{i=1}^m \mathbf{e}(c_i) \quad (3)$$

3. 以质心向量查询 ChromaDB，获取 $k + |C|$ 个候选

4. 过滤已读论文，返回前 k 个

该方法本质上是**基于内容的协同过滤**——质心方向代表用户的综合兴趣方向，实时计算，无需离线训练。

5.3 Reranker 精排

系统支持三种精排策略：

1. **无精排 (none)**：直接返回 ChromaDB 的距离排序结果。
2. **余弦相似度重排 (cosine)**：对查询和候选文档统一进行 Embedding，L2 归一化后计算余弦相似度：

$$\text{score}_i = \frac{\mathbf{q} \cdot \mathbf{d}_i}{\|\mathbf{q}\| \cdot \|\mathbf{d}_i\|} \quad (4)$$

3. **Qwen3-Rerank (qwen)**：调用 DashScope 的 `qwen3-rerank` 模型，对查询和候选文档进行交叉编码精排，效果最优但延迟较高。

5.4 PDF 全文提取与分块 Embedding

为使 LLM 能够基于论文全文内容回答问题，系统实现了 PDF 全文提取与分块 Embedding 管道：

5.4.1 全文提取

使用 PyMuPDF (`fitz`) 逐页提取 PDF 文本。对 120 篇论文 PDF 进行处理，共提取约 2080 个文本分块。

5.4.2 分块策略

采用滑动窗口分块：每块约 800 词，相邻块重叠 100 词，确保上下文连续性：

$$\text{chunks} = \{\text{words}[i : i + 800] \mid i = 0, 700, 1400, \dots\} \quad (5)$$

对超长文本（如含编码数据的 PDF），在 Embedding 前截断至 8000 字符以满足 API 输入限制。

5.4.3 向量索引

分块文本通过 `text-embedding-v4` 生成 1024 维向量，存入 ChromaDB 的 `paper_chunks` collection，每个分块附带 `paper_id` 和 `chunk_index` 元数据，支持按论文 ID 过滤检索：

```
1 collection.upsert(  
2     ids=["paper_42_chunk_3"],  
3     documents=["chunk text..."],  
4     embeddings=[...],          # 1024-dim vector  
5     metadatas=[{"paper_id": 42, "chunk_index": 3}],  
6 )
```

Listing 1: 分块索引数据结构

算法层提供以下索引/检索/删除端点：

- POST /index: 索引论文摘要级向量至 papers collection（种子导入与用户上传共用）
- POST /index-chunks: 批量索引全文分块至 paper_chunks collection
- POST /search-chunks: 检索分块，支持 paper_id 过滤（全库或单篇论文检索）
- POST /delete-paper: 按 paper_id 从 papers 与 paper_chunks 两个 collection 中删除该论文的全部向量（用户删除上传论文时调用）

5.5 LLM 学术问答与 RAG

通过 DashScope OpenAI 兼容接口调用 qwen-flash 模型，支持流式和非流式两种模式。

5.5.1 论文问答

在论文详情页，用户可直接提问。系统从该论文的全文分块中检索最相关的 5 个片段注入系统提示词，使 LLM 能基于论文实际全文内容（而非仅摘要）回答问题。

5.5.2 RAG 全库检索增强生成

在 AI 对话页面，系统以用户问题为查询从全库 2080 个分块中检索 Top-8 最相关片段，按论文 ID 聚合后注入系统提示词，并生成引用列表。LLM 回答后，前端显示参考论文标签，用户可点击跳转至对应论文详情页。

流式响应通过 FastAPI StreamingResponse 实现 SSE 推送。

6 前端界面实现

前端基于 Vue 3 Composition API + Element Plus 构建，通过 Vite 开发服务器代理/api/* 请求至业务后端。

6.1 页面组成

表 7: 前端页面与功能

页面	组件	功能说明
登录	LoginForm.vue	用户名密码登录, JWT 持久化到 local-Storage
注册	RegisterForm.vue	新用户注册 (用户名、邮箱、姓名)
论文搜索	PaperSearch.vue	语义搜索 + 搜索历史标签 + Rerank 开关 + 分页
论文详情	PaperDetail.vue	标题、摘要、PDF 阅读器、AI 论文问答
论文推荐	PaperRecommend.vue	基于浏览历史的个性化推荐列表
我的论文	MyPapers.vue	arXiv 链接导入 + 可编辑元数据表单 + 我的论文列表 (可删除)
AI 对话	Chat.vue	SSE 流式渲染 + RAG 检索增强 + 参考论文
用户管理	Profile.vue 等	个人信息、用户列表、添加用户

6.2 关键前端特性

- **路由守卫**: 通过 vue-router 的 beforeEach 钩子, 未登录用户自动重定向至登录页。
- **Token 持久化**: JWT 存储在 localStorage, Axios 拦截器自动注入 Authorization 头。
- **流式渲染**: AI 对话使用 fetch + ReadableStream 接收 SSE, 通过 marked 库实时渲染 Markdown。
- **论文问答**: 论文详情页内嵌 AI 问答框, 提供快捷提问 (“核心方法”、“主要贡献”等), 基于论文全文分块回答。
- **论文上传**: 在 “我的论文” 页粘贴 arXiv 链接即可解析出元数据并填入可编辑表单, 确认后一键导入; 导入的论文复用论文详情页阅读 (PDF + AI 问答), 列表支持删除。
- **RAG 参考引用**: AI 对话页自动检索全库论文分块, 回答后显示参考论文标签, 点击可跳转至对应详情页。
- **搜索历史**: 去重后以 Element Plus Tag 组件展示, 点击标签可快速复搜。
- **Rerank 开关**: 搜索页面提供精排开关, 设置持久化到 localStorage。
- **状态管理**: 使用 Pinia 管理用户状态, 支持 Token 自动恢复。

6.3 路由设计

表 8: 前端路由表

路径	组件	说明
/	Login.vue	登录页
/register	Register.vue	注册页
/index	Index.vue	主布局 (Header + 侧边栏)
/index/paperSearch	PaperSearch.vue	论文搜索
/index/paperDetail/:id	PaperDetail.vue	论文详情
/index/paperRecommend	PaperRecommend.vue	论文推荐
/index/myPapers	MyPapers.vue	我的论文 (上传/管理)
/index/chat	Chat.vue	AI 对话
/index/checkUserInfo	CheckUserInfo.vue	用户管理

7 算法分析与实验

7.1 实验环境

- **数据集:** 120 篇来自 OpenAlex 的真实数据库/数据管理领域论文 (含标题、摘要、关键词)
- **Embedding 模型:** text-embedding-v4 (1024 维), DashScope API
- **向量数据库:** ChromaDB (persistent 模式)
- **Ground Truth:** 基于 8 个主题类别构建 20 条搜索查询及对应相关论文标注
- **重复次数:** 每组实验重复 3 次取均值

7.2 评估指标

表 9: 评估指标定义

指标	定义
P@K	前 K 个结果中相关文档的比例
R@K	前 K 个结果覆盖的召回率
MRR	第一个相关结果的排名倒数的均值
NDCG@K	归一化折损累积增益，考虑排序位置的加权指标
MAP	各查询 Average Precision 的均值
Hit Rate@K	推荐的 K 篇中是否包含目标论文
Coverage	推荐结果覆盖的论文比例
Diversity	推荐列表内部两两余弦距离的均值
Novelty	推荐物品的平均自信息量

7.3 实验一：搜索系统消融实验

逐一替换或移除系统组件，评估各组件对搜索效果的贡献。

表 10: 搜索系统消融实验结果

编号	配置	P@5	P@10	P@20	NDCG@5	MRR	延迟 (ms)
A1	完整系统（向量 +Rerank）	0.825	0.713	0.513	0.847	0.958	467.7
A2	SQL LIKE 关键词搜索	0.542	0.508	0.396	0.546	0.690	7.3
A3	TF-IDF 余弦相似度	0.725	0.646	0.513	0.753	0.872	0.5
A4	向量检索（无 Rerank）	0.825	0.713	0.513	0.847	0.958	186.1
A5	向量检索（余弦距离）	0.825	0.713	0.513	0.847	0.958	250.1

分析：向量语义检索（A1/A4/A5）在 P@K 和 NDCG@K 上显著优于 SQL LIKE（A2），P@5 提升 52.2%（0.542→0.825）。TF-IDF（A3）效果居中但在 P@10 上弱于向量检索（0.646 vs 0.713），说明预训练 Embedding 在语义理解上有明显优势。Rerank 对 Top-K 指标无额外贡献（A1 与 A4 一致），但引入约 280ms 额外延迟。

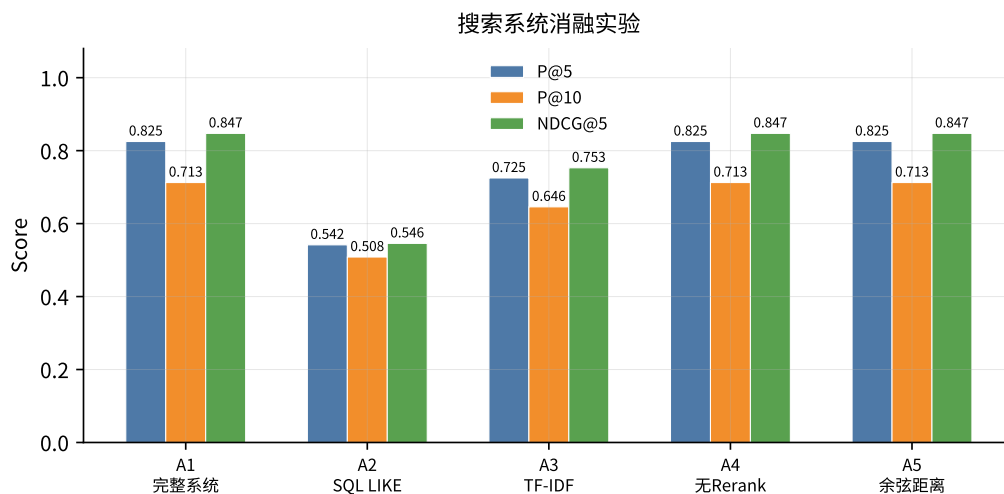


图 3: 搜索系统消融实验: 各配置在 P@5、P@10、NDCG@5 上的对比

7.4 实验二: 推荐系统消融实验

表 11: 推荐系统消融实验结果

编号	配置	Hit Rate@10	Coverage	Diversity	延迟 (ms)
B1	完整质心推荐	0.727	0.542	0.494	4.9
B2	仅最后一次点击推荐	0.682	0.658	0.508	4.8
B3	缩短窗口 (5 篇)	0.727	0.542	0.494	4.8
B4	不过滤已读	0.682	0.625	0.490	4.8
B5	质心加随机扰动	0.591	0.742	0.533	4.9

分析: 多篇点击的质心推荐 (B1) 优于仅用最后一篇 (B2), Hit Rate 提升 6.6% (0.682→0.727)。随机扰动 (B5) 牺牲了命中率但显著提高了 Coverage 和 Diversity, 体现了精准与探索的经典权衡。

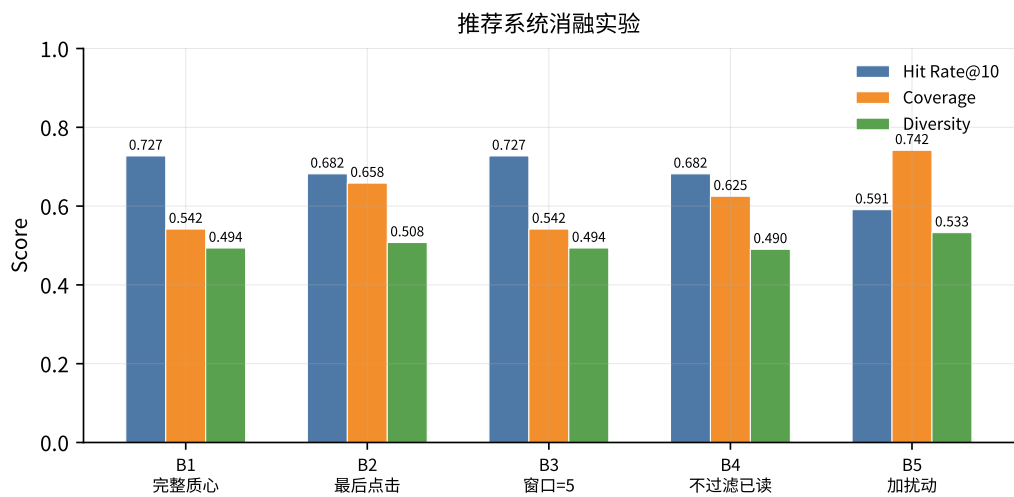


图 4: 推荐系统消融实验：各配置在 Hit Rate@10、Coverage、Diversity 上的对比

7.5 实验三：搜索系统性能评估

表 12: 搜索系统完整性能评估（3 次重复）

指标	P@5	P@10	P@20	MRR	MAP
均值	0.825	0.713	0.513	0.958	0.447

表 13: 搜索延迟分解（ms）

阶段	均值	标准差
Embedding 编码	202.2	19.5
ChromaDB 检索	6.1	0.6
Reranker 精排	8.6	0.6
端到端总计	217.0	19.0

分析：延迟瓶颈主要在 Embedding 编码（占 93.2%），ChromaDB 向量检索本身仅需约 6ms。标准差较小（19.0ms），说明系统延迟稳定。

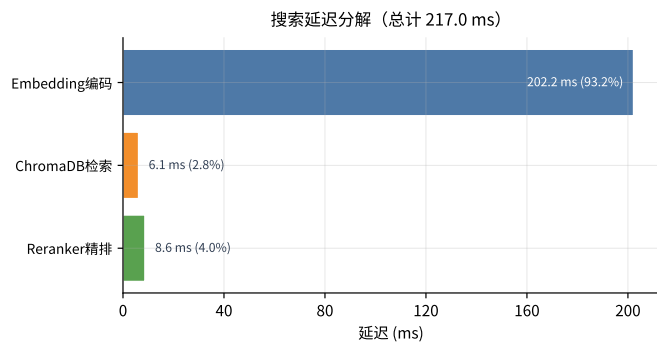


图 5: 搜索延迟分解：Embedding 编码占端到端延迟的 93.2%

7.6 实验四：推荐系统性能评估

表 14: 推荐系统性能评估

指标	Hit Rate@10	Coverage	Diversity	Novelty	延迟 (ms)
均值	0.591	0.542	0.494	6.644	4.9

7.7 实验五：距离度量对比

表 15: 不同距离度量的检索效果

距离度量	P@5	P@10	NDCG@5	NDCG@10	MRR	延迟 (ms)
L2 欧氏距离	0.825	0.713	0.847	0.760	0.958	3.64
Cosine 余弦	0.825	0.713	0.847	0.760	0.958	3.75
Inner Product	0.825	0.713	0.847	0.760	0.958	4.89

分析：在 `text-embedding-v4` 生成的归一化向量上，三种度量效果完全一致——对归一化向量，L2 距离、余弦相似度和内积在排序上数学等价。

7.8 实验六：文档表示对比

表 16: 不同文档表示方式的检索效果

编号	文档表示	P@5	P@10	NDCG@5	MRR	MAP
C1	仅 Title	0.692	0.600	0.732	0.931	0.371
C2	仅 Abstract	0.850	0.683	0.863	0.938	0.420
C3	仅 Keywords	0.767	0.629	0.807	0.958	0.381
C4	Title + Abstract	0.833	0.696	0.854	0.958	0.432
C5	Title + Abstract + Keywords	0.825	0.713	0.847	0.958	0.447

分析：仅 Abstract (C2) 在 P@5 和 NDCG@5 上最高 (0.850、0.863)，但完整拼接 (C5) 在 P@10 和 MAP 上最优 (0.713、0.447)，综合表现最均衡，故作为系统默认配置。仅 Title (C1) 效果最差，说明标题信息量有限。

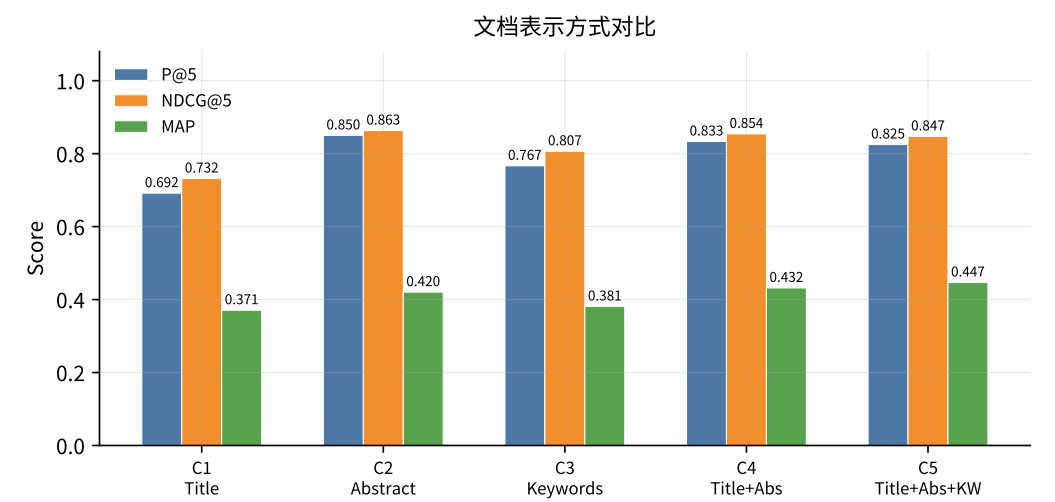


图 6: 文档表示方式对比：不同字段组合在 P@5、NDCG@5、MAP 上的表现

7.9 实验七: Reranker 对比

表 17: 不同 Reranker 策略的效果

编号	Pipeline	P@5	P@10	NDCG@5	MAP	延迟 (ms)
R1	Base Vector（无精排）	0.825	0.713	0.847	0.447	259.7
R2	Qwen3-Rerank（召回 60）	0.833	0.663	0.846	0.447	602.7
R3	Qwen3-Rerank（召回 100）	0.825	0.679	0.841	0.451	678.5

分析：Qwen3-Rerank 在 P@5 上微幅提升（R2=0.833 vs R1=0.825），但 P@10 反而下降（0.663 vs 0.713），且延迟翻倍（~603ms vs 260ms）。扩大召回窗口（R3 vs R2）效果无明显差异，说明 60 的召回量已足够。在当前数据集上，Rerank 的精度增益有限，但延迟代价显著。

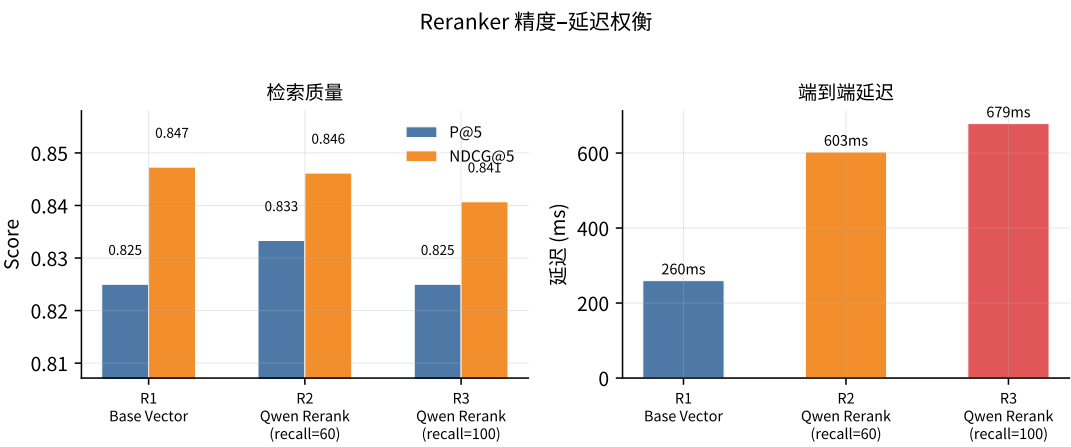


图 7: Reranker 精度-延迟权衡：精排提升 P@5 约 2.5%，但延迟增加约一倍

7.10 实验八：数据规模扩展测试

表 18: 不同数据规模下的性能

论文数	索引构建 (ms)	Embedding(ms)	平均查询 (ms)	P95(ms)	Max(ms)
20	1,287	1,176	2.60	3.56	4.42
40	2,818	2,682	2.76	4.03	5.24
60	4,037	3,876	3.66	8.62	11.41
80	5,212	4,960	2.80	4.18	5.48
100	6,376	6,179	2.89	4.44	5.89
120	8,228	7,994	2.84	4.58	6.27

分析：索引构建时间与论文数量呈线性关系，瓶颈在 Embedding 编码（API 调用），ChromaDB upsert 仅占约 3%。查询延迟在 120 篇规模下稳定在 2.6–3.7ms（均值），表明系统具有良好的可扩展性。

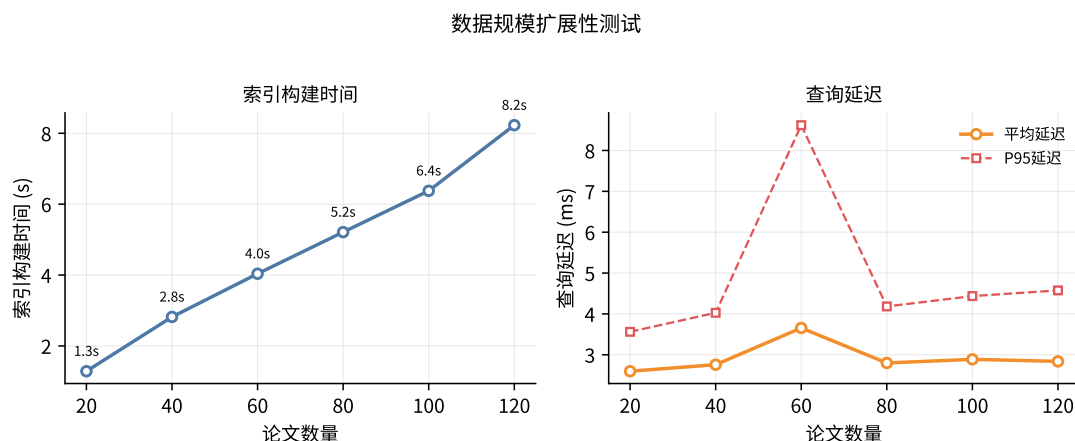


图 8: 数据规模扩展性测试：索引构建时间线性增长，查询延迟稳定在 2.6–3.7ms

7.11 实验小结

1. 向量语义检索相比 SQL LIKE，P@5 提升 52.2% (0.542 → 0.825)，语义理解能力显著。
2. 质心推荐比仅用最后一篇点击 Hit Rate 提升 6.6% (0.682→0.727)，多篇历史有助于捕捉综合兴趣。
3. 系统端到端搜索延迟约 217ms，瓶颈在 Embedding 编码 (93.2%)，ChromaDB 检索本身仅约 6ms。
4. Qwen3-Rerank 在当前数据集上精度增益有限 (P@5+0.8%)，但延迟增加约一倍，适合对精度要求极高的场景。
5. 归一化向量下 L2/Cosine/IP 三种度量效果完全等价。

6. 完整的 Title+Abstract+Keywords 拼接在 P@10 和 MAP 上最优，是最均衡的文档表示方式。

8 测试数据

8.1 论文数据集

系统使用 120 篇来自 OpenAlex 的真实学术论文作为种子数据，涵盖数据库与数据管理领域的多个方向：

- 查询优化 (Query Optimization)
- SQL 索引 (SQL Indexing)
- 分布式数据库 (Distributed Database)
- 事务处理 (Transaction Processing)
- 图数据库 (Graph Database)
- 云数据库 (Cloud Database)
- 安全与隐私 (Security & Privacy)
- 信息检索与索引 (Retrieval & Indexing)

每篇论文包含标题、完整摘要、作者列表、发表会议/期刊、年份、关键词和来源链接，数据来源为 OpenAlex 学术 API。此外，系统本地存储了 120 篇论文的 PDF 全文 (约 187MB)，通过 PyMuPDF 提取文本后分块 Embedding，共生成约 2080 个全文分块向量，用于论文问答和 RAG 检索增强。

8.2 Ground Truth 构建

搜索实验的 Ground Truth 基于主题规则自动生成：

1. 定义 8 个主题类别，每个类别包含若干关键词规则。
2. 设计 20 条接近真实用户意图的搜索查询 (非简单论文标题复制)。
3. 论文按摘要和关键词与主题规则的匹配度标注为相关/不相关。
4. 每条查询标注关联的 focus 关键词，用于生成分级相关度。

8.3 推荐评估数据

推荐实验的评估数据基于模拟用户行为构建：

1. 按主题类别将论文分组，为每个主题模拟 2-3 个用户 Session。
2. 每个 Session 包含 5-10 次点击历史 (同主题论文)。
3. 使用 Leave-One-Out 方法：保留最后一篇作为目标，用前 $n - 1$ 篇生成推荐。

9 用户手册

9.1 环境要求

- Python 3.10+
- Node.js 18+
- MySQL 8.0+
- DashScope API Key（用于 Embedding 和 LLM）

9.2 安装与配置

```
1 # 克隆仓库
2 git clone git@github.com:JunjieNian/paper-search-database.git
3 cd paper-search-database
4
5 # Python 环境
6 python3 -m venv .venv
7 source .venv/bin/activate
8 pip install -r backend/requirements.txt
9 pip install -r backend_algo/requirements.txt
10
11 # 前端依赖
12 cd frontend && npm install && cd ..
```

Listing 2: 安装依赖

```
1 cp backend/.env.example backend/.env
2 cp backend_algo/.env.example backend_algo/.env
3 cp frontend/.env.development.example
  frontend/.env.development.local
4
5 # 编辑 backend/.env: 填写 MYSQL_USER, MYSQL_PASSWORD
6 # 编辑 backend_algo/.env: 填写 DASHSCOPE_API_KEY
```

Listing 3: 配置环境变量

```
1 CREATE DATABASE paper_search CHARACTER SET utf8mb4;
```

Listing 4: 创建数据库

9.3 启动服务

方式一：一键启动（推荐）

```
1 bash start_all.sh      # 启动全部服务
2 bash stop_all.sh      # 停止全部服务
```

方式二：手动启动

```
1 # 终端1：算法层
2 cd backend_algo && python -m uvicorn main:app --host 0.0.0.0
   --port 8003
3
4 # 终端2：业务后端
5 cd backend && python -m uvicorn main:app --host 0.0.0.0 --port
   8000
6
7 # 终端3：前端
8 cd frontend && npm run dev -- --host 0.0.0.0
```

9.4 导入种子数据

```
1 cd backend && python seed_papers.py      #
   导入论文元数据+摘要索引
2 cd backend && python extract_chunks.py    # PDF全文提取+分块索引
```

seed_papers.py 将 120 篇论文写入 MySQL 并在 ChromaDB 中建立摘要级向量索引。extract_chunks.py 使用 PyMuPDF 提取 PDF 全文，按 800 词/块分块后通过算法层/index-chunks 建立全文分块索引（约 2080 个 chunks），支持后续的论文问答和 RAG 检索。

9.5 功能使用

1. **注册与登录：**打开 <http://localhost:5173>，点击注册创建账户，然后登录。
2. **论文搜索：**在侧边栏点击“论文搜索”，输入自然语言查询（如“query optimization in distributed databases”），点击搜索。支持切换是否启用 Rerank 精排。
3. **查看论文：**点击搜索结果中的论文标题，查看论文详情（标题、作者、摘要、PDF 全文）。
4. **论文问答：**在论文详情页底部的“AI 论文问答”区域，可以直接就该论文提问（如“核心方法”、“主要贡献”），AI 基于论文全文分块回答。

5. **上传我的论文:** 点击侧边栏”我的论文”, 粘贴 arXiv 论文链接 (如 <https://arxiv.org/abs/2310.> 并点击”解析”, 系统自动提取标题、作者、摘要等信息填入表单; 确认或微调后点击”确认导入”, 论文即加入个人论文库, 可像系统论文一样阅读、问答, 并出现在你 (且仅你本人) 的搜索与推荐结果中; 列表中可随时删除已上传论文。
6. **论文推荐:** 浏览若干论文后, 点击侧边栏”论文推荐”, 系统将根据浏览历史推荐相关论文。
7. **AI 对话:** 点击侧边栏”AI 对话”, 与 LLM 进行多轮学术问答。系统自动检索全库论文全文分块 (RAG), 回答后显示参考论文标签, 点击标签可跳转至对应论文详情。
8. **搜索历史:** 搜索页面上方显示去重后的搜索历史标签, 点击可快速复搜。

9.6 访问地址

服务	地址
前端界面	http://localhost:5173
业务层 API 文档	http://localhost:8000/docs
算法层 API 文档	http://localhost:8003/docs

9.7 运行实验

```

1 cd experiments
2 pip install -r requirements.txt
3 python run_all.py # 运行实验 1-8
4 RUN_CONCURRENT=1 python run_all.py # 包含并发压测

```

实验结果以 JSON 格式保存在 `experiments/results/` 目录。

10 项目结构

```

mysql_fastapi_vue_project/
|-- frontend/ # 前端 (Vue 3 + Element Plus)
|   |-- src/
|       |-- router/index.ts # 路由 + 登录守卫
|       |-- store/user.ts # Pinia 状态管理
|       |-- request/api.ts # API 接口封装
|       |-- components/ # 业务组件
|       |-- pages/ # 页面组件
|       |-- vite.config.ts # 构建配置 + 代理

```

```

|
|-- backend/                                # 业务层 (FastAPI + MySQL)
|   |-- main.py                             # 路由 + JWT认证 + RAG对话
|   |-- models.py                           # ORM 模型 (4张表)
|   |-- schemas.py                          # Pydantic 校验
|   |-- crud.py                             # CRUD 操作
|   |-- security.py                         # 密码哈希
|   |-- seed_papers.py                      # 种子数据导入
|   |-- extract_chunks.py                   # PDF全文提取+分块索引
|   |-- paper_ingest.py                     # arXiv导入:
|       OpenAlex元数据+PDF下载+分块
|   |-- pdf_store/                          # 论文PDF文件 (含用户上传)
|   |-- seed_data/papers.json               # 120篇论文数据
|
|-- backend_algo/                           # 算法层 (FastAPI + ChromaDB)
|   |-- main.py                             # 搜索/推荐/对话/分块检索路由
|   |-- vector_store.py                     # ChromaDB 交互 (papers+chunks)
|   |-- reranker.py                         # Reranker 实现
|   |-- config.py                           # 配置加载
|
|-- experiments/                             # 算法分析实验 (8组)
|   |-- run_all.py                          # 一键运行
|   |-- metrics.py                          # 评估指标
|   |-- ground_truth.py                     # Ground Truth
|   |-- 01_ablation/                        # 消融实验
|   |-- 02_performance/                     # 性能评估
|   |-- 03_query_optimization/               # 查询优化对比
|   |-- 04_scalability/                     # 可扩展性测试
|   |-- results/                             # 实验结果 (JSON)
|
|-- report/                                 # 课程PJ报告 (LaTeX)
|-- start_all.sh / stop_all.sh              # 一键启停
|-- README.md                               # 项目文档

```

11 总结

本项目完整实现了一套学术论文搜索与推荐系统，严格遵循课程要求的三层分层架构（Vue 界面层 + FastAPI 业务层 + FastAPI 算法层）。系统主要成果包括：

1. **数据库设计**：设计了包含 `users`、`papers`、`user_clicks`、`search_history` 四张表的关系数据库，支持用户管理、论文存储和行为追踪；其中 `papers.owner_id` 自

引用 `users` 表，实现论文归属标记与”个人/公共”论文的可见性隔离。

2. **业务层实现：**基于 FastAPI 实现了完整的 RESTful API，包括 JWT 认证、论文 CRUD、搜索/推荐代理、PDF 管理、三模式 LLM 对话（论文问答/RAG/纯对话），以及用户论文上传（arXiv 链接导入 OpenAlex 元数据 + arxiv.org 下载 PDF + 双重索引）与基于 `owner_id` 的全链路可见性隔离，并实现了算法层不可用时 SQL 降级机制。
3. **算法层实现：**实现了基于 ChromaDB 的向量语义检索、Embedding 质心推荐、多种 Reranker 精排策略、PDF 全文提取与分块 Embedding（约 2080 个 chunks），以及基于全文分块的 RAG 检索增强生成。
4. **前端界面：**基于 Vue 3 + Element Plus 构建了包含搜索、推荐、详情、我的论文、对话等 8 个页面的完整前端，支持流式 Markdown 渲染、论文详情页内嵌 AI 问答、RAG 对话参考论文引用标签、arXiv 链接上传论文、搜索历史等特性。
5. **算法实验：**设计并执行了 8 组完整实验，涵盖消融分析、性能评估、查询优化和可扩展性测试，验证了向量检索相比关键词搜索 P@5 提升 52.2%（0.542→0.825）、推荐系统 Hit Rate 达 0.727、端到端搜索延迟约 217ms 等核心指标。